

Minimum Weighted Completion Time Scheduling

Petar Dinić
mi17090@alas.matf.bg.ac.rs

Contents

1	Uvod	2
2	Opis rešenja	3
2.1	Reprezentacija podataka	3
2.2	Gruba sila	4
2.3	Pohlepni algoritmi	4
2.4	S-metaheuristike	4
2.5	Genetski algoritam	5
2.6	Memetički algoritam	6
3	Eksperimenti i analiza rezultata	6
3.1	Brute force naspram pohlepnih heuristika	7
3.2	S metaheuristike	8
3.3	Populacioni algoritmi	9
4	Zaključak	10

1 Uvod

Problem rasporedjivanja zadataka na procesore predstavlja jedan od fundamentalnih problema optimizacije, sa širokom primenom u oblastima kao što su planiranje proizvodnje, rasporedjivanje procesa u operativnim sistemima i upravljanje resursima u distribuiranim sistemima.

U ovom radu razmatra se varijanta problema poznata kao Minimum Weighted Completion Time, MWCT, definisana na sledeći način:

- Dat je skup od n nezavisnih zadataka $T = \{t_1, t_2, \dots, t_n\}$ i m identičnih paralelnih procesora.
- Svaki zadatak t_i karakterišu tri parametra: vreme puštanja u rad (*release time*) r_i , trajanje l_i i težina (prioritet) w_i .
- Zadatak t_i ne sme početi sa izvršavanjem pre trenutka r_i , ne sme se prekidati (bez preemptivnosti), a svaki procesor može u jednom trenutku izvršavati najviše jedan zadatak.
- Cilj je odrediti raspored (dodelu zadataka procesorima i njihov redosled izvršavanja na svakom procesoru) koji minimizuje ukupno težinsko vreme završetka:

$$\min \sum_{i=1}^n w_i C_i$$

gde je C_i vreme završetka zadatka t_i u datom rasporedu.

Problem je NP-težak pa je za veće instance neophodno posegnuti za heurističkim i metaheurističkim pristupima, budući da egzaktno rešavanje (metoda grube sile) postaje neizvodljivo već za relativno mali broj zadataka (U našem slučaju za dimenziju 12).

2 Opis rešenja

2.1 Reprezentacija podataka

Instanca problema je predstavljena parom (m, T) gde je m broj procesora, a T lista zadataka. Svaki zadatak je predstavljen strukturom sa četiri polja: identifikator, vreme puštanja u rad, trajanje i težina.

```
@dataclass
class Task:
    id: int
    r: int # release time
    l: int # duration (length)
    w: int # weight

@dataclass
class Instance:
    n_processors: int
    tasks: list[Task]
```

Instance se generišu nasumično (`generate_instance`), sa mogućnošću podešavanja opsega vrednosti za r_i , l_i i w_i , kao i (opciono) korelacije između trajanja i težine zadatka (nezavisno, pozitivno ili negativno korelisano), radi ispitivanja ponašanja algoritama na različitim profilima instanci.

Rešenje (raspored) predstavljeno je kao lista od m lista zadataka, jedna lista po procesoru, sa zadacima poredjanim redosledom izvršavanja na tom procesoru:

```
Solution = list[list[Task]]
```

Ciljna funkcija se računa:

```
def objective(schedule):
    total = 0
    for processor in schedule:
        current_time = 0
        for task in processor:
            start = max(current_time, task.r)
            completion = start + task.l

            total += task.w * completion
            current_time = completion
    return total
```

Za rešavanje ovog problema korišćemo različite algoritme kao što su simulirano kaljenje, VNS, genetski algoritam i memetički algoritam.

2.2 Gruba sila

Kao referentna vrednost za procenu kvaliteta heurističkih rešenja, implementiran je egzaktan algoritam koji ispituje *sve* permutacije zadataka ($n!$ permutacija), dekodira svaku permutaciju u konkretan raspored (svaki naredni zadatak se dodeljuje procesoru koji ga može najranije pokrenuti) i pamti raspored sa najmanjom vrednošću ciljne funkcije. Zbog faktorijske složenosti $O(n! \cdot n \cdot m)$, ovaj pristup je upotrebljiv samo za vrlo male instance u našim merenjima, već za $n = 10$ zadataka izvršavanje traje oko 130 sekundi na jednoj instanci.

2.3 Pohlepni algoritmi

Implementirane su dve varijante pohlepne konstrukcije rasporeda, zasnovane na strategiji *liste raspoređivanja*: procesori se obrađuju redom po trenutku kada se oslobadaju (pomoću strukture heap), a u svakom koraku bira se zadatak sa najvećom vrednošću odnosa $w_i/(t + l_i)$, gde je t trenutak u kome procesor postaje slobodan.

Dve varijante se razlikuju po tome kako biraju zadatak kada nijedan zadatak još nije "pušten u rad" (tj. $r_i > t$ za sve preostale zadatke):

- **Greedy:** procesor čeka do trenutka najranijeg budućeg puštanja u rad i tada bira najbolji zadatak među onima koji su tada raspoloživi.
- **Modified Greedy:** svaki zadatak se boduje već sada (optimistički, koristeći njegovo sopstveno r_i u formuli), pa je moguće da procesor "sačeka" izuzetno atraktivan budući zadatak.

2.4 S-metaheuristike

Lokalna pretraga (Local Search). Koristi dve susedstva:

- **Swap:** zamena mesta dva zadatka (na istom ili različitim procesorima),
- **Relocate:** premeštanje jednog zadatka na drugi procesor (na proizvoljnu poziciju).

Pretraga koristi strategiju (*first-improvement*): Polazi od rešenja koje smo dobili **modified_greedy** algoritmom i u svakoj iteraciji primenjuje se prvi pronadjeni potez koji strogo smanjuje vrednost ciljne funkcije, sve dok se ne dostigne lokalni optimum (nijedan potez iz posmatranih susedstava više ne poboljšava rešenje).

Promenljiva okolina (Variable Neighborhood Search, VNS): Kombinuje shaking rastuće jačine $k = 1, \dots, k_{max}$ sa lokalnom pretragom: iz trenutno najboljeg rešenja generiše se slučajno poremećeno rešenje jačine k , nad njim se primenjuje lokalna pretraga, i ukoliko je dobijeno rešenje bolje od trenutno najboljeg, ono postaje novo najbolje rešenje i pretraga se vraća na jačinu $k = 1$; u suprotnom, jačina poremećaja se povećava. Jačina poremećaja je definisana brojem slučajnih poteza (za $k \leq 5$) ili potpunim slučajnim mešanjem svih zadataka (za $k > 5$, najjača varijanta).

Simulirano kaljenje (Simulated Annealing): Polazi od greedy rešenja i u svakoj iteraciji bira slučajan potez (zamenu ili premeštanje). Potez koji poboljšava rešenje se uvek prihvata; potez koji pogoršava rešenje prihvata se sa verovatnoćom $e^{-\Delta/T}$, gde je Δ pogoršanje ciljne funkcije, a T trenutna "temperatura". Temperatura opada geometrijski nakon fiksnog broja iteracija na datoj temperaturi, čime se pretraga postepeno usmerava ka eksploataciji najboljeg pronadjenog regiona.

2.5 Genetski algoritam

Za razliku od S-metaheuristika koje održavaju jedno rešenje, genetski algoritam paralelno evoluira populaciju rešenja.

Reprezentacija (hromozom): Hromozom je permutacija identifikatora zadataka, a ne direktna reprezentacija rasporeda. Ovakav izbor obezbeđuje da je svaka jedinka *automatski izvodljiva* nakon ukrštanja i mutacije svaki zadatak se u permutaciji pojavljuje tačno jednom, pa nema rizika da neki zadatak bude izostavljen ili dupliran, što bi direktna reprezentacija rasporeda zahtevala da se rešava dodatnom logikom "popravke" hromozoma.

Dekodiranje Permutacija se pretvara u konkretan raspored istom strategijom liste rasporedjivanja koja se koristi i kod grube sile: zadaci se obrađuju redosledom iz hromozoma, a svaki se dodeljuje procesoru na kome može najranije da počne izvršavanje (uz poštovanje r_i).

Genetski operatori.

- **Selection:** turnirska selekcija (veličina turnira $k = 3$).
- **Crossover:** kopira se slučajan segment od prvog roditelja, a preostale pozicije popunjava genima drugog roditelja, poštujući njihov relativni redosled. Garantuje validnu permutaciju bez dodatne popravke.
- **Mutation:** zamena mesta dva slučajno izabrana zadatka u permutaciji (*swap mutation*).
- **Elitism:** najbolje jedinke iz generacije se direktno prenose u sledeću generaciju, bez izmena.

2.6 Memetički algoritam

Memetički algoritam predstavlja proširenje genetskog algoritma u kome se, pored evolutivnih operatora, koristi i lokalna pretraga za dodatno poboljšavanje jedinki. Na ovaj način se kombinuju sposobnost genetskog algoritma da istražuje veliki prostor rešenja i sposobnost lokalne pretrage da detaljno optimizuje pojedinačna rešenja.

Enkodiranje: Pored dekodiranja, pošto local search ne radi sa permutacijom već sa postojećim rasporedom, potrebno nam je da enkodiramo takav raspored nazad u hromozom, odnosno permutaciju identifikatora zadataka.

Genetski operatori: Memetički algoritam koristi iste osnovne genetske operatore kao i genetski algoritam.

Lokalna pretraga: Glavna razlika u odnosu na standardni genetski algoritam je primena lokalne pretrage na odabrane jedinke populacije. U svakoj generaciji lokalnom pretragom se obavezno poboljšavaju najbolje `ls_elitism` jedinke, dok se preostale jedinke biraju za lokalnu pretragu sa verovatnoćom `local_search_rate`. Pre primene lokalne pretrage hromozom se dekodira u konkretan raspored.

Lokalna pretraga pokušava da pronadje bolje rešenje u okolini trenutnog rasporeda. Ukoliko pronadjeno rešenje ima manju vrednost funkcije cilja od originalnog rešenja, poboljšani raspored se ponovo kodira u hromozom i vraća u populaciju. Ukoliko poboljšanje nije pronadjeno, originalna jedinka ostaje nepromenjena.

Algoritam počinje generisanjem početne populacije nasumičnih hromozoma. U svakoj generaciji najpre se izračunava vrednost funkcije cilja za sve jedinke. Zatim se na odabrane jedinke primenjuje lokalna pretraga. Nakon lokalnog poboljšanja određuje se trenutno najbolje rešenje i ažurira se najbolje pronadjeno rešenje ukoliko je došlo do poboljšanja.

Nakon toga se formira nova populacija. Najboljih *elitism* jedinki se direktno prenosi u novu generaciju, dok se ostatak populacije dobija turnirskom selekcijom roditelja, ukrštanjem i mutacijom. Ovaj postupak se ponavlja zadati broj generacija.

Na taj način genetski algoritam obezbeđuje globalno istraživanje prostora rešenja, dok lokalna pretraga dodatno iskorišćava kvalitet obećavajućih jedinki. Kombinacija ova dva pristupa čini osnovnu karakteristiku memetičkog algoritma.

3 Eksperimenti i analiza rezultata

Svi eksperimenti su izvršeni nad instancama dimenzija $n \in \{4, 5, \dots, 25\}$, gde n predstavlja broj taskova u instanci. Za svaku dimenziju generisano je po

10 instanci, a prikazane vrednosti prosečne funkcije cilja (*average_objective*) i vremena izvršavanja predstavljaju **prosek preko tih 10 instanci**. Ovo je bitno napomenuti jer pojedinačne instance mogu znatno da variraju u težini, pa prosek uprosečava i "lakše" i "teže" slučajeve date dimenzije.

3.1 Brute force naspram pohlepnih heuristika

Tabela 1 upoređuje algoritam grube sile (*brute force*) sa dve konstruktivne heuristike, pohlepnom (*greedy*) i modifikovanom pohlepnom (*modified greedy*) algoritmom.

Brute Force vs. Greedy vs. Modified Greedy

Dim	Brute Force Objective	Brute Force Time (s)	Greedy Objective	Greedy Time (s)	Modified Greedy Objective	Modified Greedy Time (s)
4	38.61	0.0134	39.20	0.0024	39.57	0.0017
5	76.45	0.0465	78.58	0.0020	78.47	0.0019
6	139.08	0.1817	143.43	0.0024	143.86	0.0023
7	175.68	1.3531	181.42	0.0029	181.12	0.0027
8	278.29	12.0389	287.79	0.0032	290.40	0.0031
9	389.93	119.3073	407.89	0.0038	405.72	0.0036
10	550.87	1310.5814	575.75	0.0042	583.18	0.0039
11	751.73	15743.6070	786.19	0.0047	788.28	0.0044
12	—	—	938.16	0.0051	942.89	0.0049
13	—	—	1235.43	0.0055	1236.50	0.0052
14	—	—	1491.52	0.0060	1499.28	0.0057
15	—	—	1995.94	0.0062	2013.44	0.0062
16	—	—	2360.32	0.0070	2361.94	0.0067
17	—	—	2972.90	0.0077	2937.84	0.0071
18	—	—	3608.50	0.0079	3601.14	0.0076
19	—	—	3810.95	0.0085	3802.89	0.0082
20	—	—	4807.70	0.0089	4808.36	0.0236
21	—	—	5534.03	0.0095	5573.86	0.0095
22	—	—	6584.62	0.0101	6609.90	0.0098
23	—	—	7252.74	0.0105	7236.97	0.0103
24	—	—	8756.89	0.0116	8736.58	0.0106
25	—	—	10378.37	0.0119	10368.89	0.0114

— : brute force not computed beyond dimension 11 (exponential runtime, 15.743s at $n=11$)

Figure 1: Poređenje brute force, greedy i modified greedy algoritama (prosek 10 instanci po dimenziji).

Brute force algoritam daje optimalno rešenje, ali njegova vremenska složenost raste eksponencijalno sa brojem taskova: vreme izvršavanja skače sa 1310.58 s za $n = 10$ na 15743.61 s (preko 4 sata i 20 minuta) za $n = 11$, odnosno raste više od 12 puta pri uvećanju dimenzije za samo jedan task. Zbog toga rezultati za $n > 11$ nisu dostupni (u tabeli označeni sa "—") – vreme izvršavanja bi bilo praktično neupotrebljivo.

Nasuprot tome, obe pohlepne heuristike izvršavaju se u zanemarljivom vremenu (< 0.012 s) nezavisno od dimenzije, ali po ceni kvaliteta rešenja. U opsegu $n = 4-11$, gde postoji egzaktno rešenje za poredjenje, *greedy* algoritam u proseku odstupa od optimuma za 3.48%, dok *modified greedy* odstupa za 3.85%. Zanimljivo je da modifikovana varijanta ne daje konzistentno bolje rezultate od osnovnog pohlepnog algoritma – za pojedine dimenzije ($n = 5, 7, 9, 11$) daje čak nešto bolje rešenje, dok za druge ($n = 4, 6, 8, 10$) daje lošije, što ukazuje da modifikacija menja kriterijum izbora na način koji nije uniformno superioran, već zavisi od strukture instance.

3.2 S metaheuristike

Tabela 2 upoređuje brute force sa tri metaheuristike zasnovane na lokalnoj pretrazi: *local search*, *Variable Neighborhood Search* (VNS) i *simulated annealing* (SA).

Brute Force vs. Local Search vs. VNS vs. Simulated Annealing

Dim	Brute Force Objective	Brute Force Time (s)	Local Search Objective	Local Search Time (s)	VNS Objective	VNS Time (s)	Simulated Annealing Objective	Simulated Annealing Time (s)
4	38.61	0.0134	38.65	0.0034	38.61	0.0161	38.61	1.6988
5	76.45	0.0465	76.59	0.0053	76.45	1.6604	76.45	1.8270
6	139.08	0.1817	139.66	0.0078	139.08	2.7657	139.08	1.9360
7	175.68	1.3531	175.96	0.0130	175.68	5.0924	175.68	2.1051
8	278.29	12.0389	278.93	0.0199	278.29	7.5770	278.50	2.2252
9	389.93	119.3073	391.06	0.0293	389.93	10.9567	390.28	2.2863
10	550.87	1310.5814	553.01	0.0417	550.87	15.0959	551.84	2.3561
11	751.73	15743.6070	755.87	0.0540	751.73	20.2174	753.16	2.4525
12	—	—	897.38	0.0846	894.71	28.4989	897.46	2.6566
13	—	—	1172.08	0.1142	1168.03	37.6321	1171.47	2.7416
14	—	—	1421.42	0.1444	1416.70	47.2526	1422.20	2.8673
15	—	—	1902.92	0.1827	1893.97	59.2911	1900.57	2.9893
16	—	—	2221.18	0.2335	2207.68	73.3687	2217.19	3.0948
17	—	—	2773.36	0.2875	2760.36	88.6890	2771.39	3.2241
18	—	—	3385.55	0.3714	3366.46	109.8720	3379.44	3.3259
19	—	—	3579.70	0.4588	3561.78	131.3827	3577.80	3.5064
20	—	—	4530.83	0.5781	4507.69	157.8035	4528.62	3.6006
21	—	—	5239.06	0.6647	5216.84	184.2713	5242.26	3.6798
22	—	—	6223.54	0.8601	6194.09	217.1846	6222.59	3.8104
23	—	—	6748.81	1.0145	6711.07	257.8827	6742.63	3.8569
24	—	—	8164.74	1.2116	8119.08	302.5759	8153.11	4.0470
25	—	—	9651.18	1.4341	9601.93	335.0895	9645.87	4.1243

— : brute force not computed beyond dimension 11 (exponential runtime, 15.743s at $n=11$)

Figure 2: Poredjenje brute force, local search, VNS i simulated annealing algoritama (prosek 10 instanci po dimenziji).

Sve tri metaheuristike znatno prevazilaze pohlepne heuristike po kvalitetu rešenja. **VNS** u opsegu $n = 4-11$ pronalazi optimalno rešenje u svih 8 testiranih dimenzija, dok *local search* u proseku odstupa svega 0.29%, a *simulated annealing* 0.066% od optimuma. Ovo je značajno poboljšanje u odnosu na pohlepne konstrukcije, uz i dalje prihvatljivo vreme izvršavanja: za najveću testiranu dimenziju ($n = 25$) VNS-u je potrebno 335.09 s, dok je *simulated annealing* znatno

brži – 4.12 s – uz rešenje vrlo blisko VNS-ovom (9645.87 naspram 9601.93). *Local search* je pritom najbrži od tri metode (1.43 s na $n = 25$), što je očekivano jer izvršava samo jedan prolaz do lokalnog optimuma bez mehanizma izlaska iz njega, za razliku od VNS-a (koji sistematski menja susedstva) i SA-a (koji prihvata i pogoršanja rešenja sa određenom verovatnoćom).

Ovi rezultati ukazuju na jasan *trade-off* između kvaliteta rešenja i vremena izvršavanja: VNS daje najbolji kvalitet po ceni najdužeg vremena izvršavanja (na $n = 25$ oko $78\times$ sporiji od SA), dok je *local search* najbrži, ali sa najvećim odstupanjem od optimuma među ove tri metode.

3.3 Populacioni algoritmi

Tabela 3 upoređuje brute force sa *genetic algorithm* (GA), *memetic algorithm* (MA, GA obogaćen lokalnom pretragom) i VNS-om, uzetim kao referentna vrednost iz prethodne tabele.

Brute Force vs. Genetic Algorithm vs. Memetic Algorithm vs. VNS

dim	Brute Force Objective	Brute Force Time (s)	Genetic Algorithm Objective	Genetic Algorithm Time (s)	Memetic Algorithm Objective	Memetic Algorithm Time (s)	VNS Objective	VNS Time (s)
4	38.61	0.0134	38.61	6.9539	38.61	9.0853	38.61	0.9161
5	76.45	0.0465	76.45	7.5138	76.45	10.7907	76.45	1.6604
6	139.08	0.1817	139.09	8.0895	139.08	12.9793	139.08	2.7657
7	175.68	1.3531	175.70	9.1150	175.68	17.7877	175.68	5.0924
8	278.29	12.0389	278.29	9.8215	278.29	21.4744	278.29	7.5770
9	389.93	119.3073	389.98	10.5123	389.93	26.2745	389.93	10.9567
10	550.87	1310.5814	551.04	11.0442	550.87	32.0118	550.87	15.0959
11	751.73	15743.6070	751.91	11.6254	751.73	38.7419	751.73	20.2174
12	—	—	894.95	13.2486	894.71	50.3029	894.71	28.4989
13	—	—	1168.22	13.7367	1168.03	60.9948	1168.03	37.6321
14	—	—	1417.67	14.2351	1416.70	73.2457	1416.70	47.2526
15	—	—	1895.47	14.8272	1893.97	86.6539	1893.97	59.2911
16	—	—	2209.48	15.4622	2207.68	103.3739	2207.68	73.3687
17	—	—	2764.09	16.1110	2760.41	120.4213	2760.36	88.6890
18	—	—	3368.05	16.9260	3366.46	142.0086	3366.46	109.8720
19	—	—	3565.38	18.6624	3561.77	168.1432	3561.78	131.3827
20	—	—	4512.67	19.4694	4507.69	195.3716	4507.69	157.8035
21	—	—	5226.13	20.3717	5216.83	224.2236	5216.84	184.2713
22	—	—	6203.80	21.0770	6193.99	256.6015	6194.09	217.1846
23	—	—	6729.35	22.0356	6710.98	289.0848	6711.07	257.8827
24	—	—	8135.53	22.4839	8119.01	327.9652	8119.08	302.5759
25	—	—	9618.14	23.1872	9601.85	369.0303	9601.93	335.0895

— : brute force not computed beyond dimension 11 (exponential runtime, 15,743s at $n=11$)

Figure 3: Poredjenje brute force, genetic algorithm, memetic algorithm i VNS algoritama (prosek 10 instanci po dimenziji).

Memetic algorithm pokazuje najbolje rezultate od sve tri metode: u opsegu $n = 4$ –11 dostiže optimalno rešenje u svakoj testiranoj dimenziji (prosečno odstupanje 0.00%), identično kao VNS, dok čist *genetic algorithm* u proseku odstupa svega 0.011% od optimuma. Iako je razlika u kvalitetu između GA i MA mala, ona direktno potvrđuje očekivani efekat dodavanja lokalne pretrage u GA. Dosledno gura populaciju ka optimalnim ili skoro optimalnim rešenjima,

na račun dodatnog vremena izvršavanja: na $n = 25$, MA-u je potrebno 369.03 s naspram 23.19 s za čist GA – oko $16\times$ sporije, jer se lokalna pretraga poziva nad delom populacije u svakoj generaciji.

Poredeći MA sa VNS-om, oba dostižu praktično identičan kvalitet rešenja (9601.85 naspram 9601.93 na $n = 25$), ali uz sličnu vremensku cenu (369.03 s naspram 335.09 s). Čist *genetic algorithm*, iako neznatno lošijeg kvaliteta, ostaje znatno brži od obe metode (23.19 s na $n = 25$) i predstavlja razuman kompromis kada je vreme izvršavanja kritičnije od poslednjeg procenta optimalnosti.

4 Zaključak

Rezultati potvrđuju očekivanu hijerarhiju pristupa problemu minimizacije težinskog vremena završetka:

- **Brute force** garantuje optimalnost, ali je upotrebljiv samo za vrlo male instance ($n \leq 11$) zbog eksponencijalne složenosti.
- **Pohlepne heuristike** (greedy, modified greedy) su ekstremno brze, ali sa najvećim odstupanjem od optimuma ($\sim 3.5\text{--}3.9\%$).
- **Metaheuristike zasnovane na lokalnoj pretrazi** (local search, SA) postižu znatno bolji kvalitet ($< 0.3\%$ odstupanja), uz umeren do visok utrošak vremena. VNS dostiže najbolji ili blizu najboljeg kvalitet rešenja.
- **Populacioni algoritmi** (GA, MA) dostižu najbolji ili blizu najboljeg kvalitet rešenja, gde MA (GA + lokalna pretraga) dosledno nadmašuje čist GA po kvalitetu, po ceni dodatnog vremena izvršavanja.

Za veće instance ($n > 11$), gde egzaktno poredjenje sa optimumom nije moguće, VNS i memetic algorithm ostaju najkonzistentniji izbor kada je kvalitet rešenja prioritet, dok su simulated annealing i genetic algorithm bolji izbor kada je vreme izvršavanja ograničavajući faktor.